

Lab 2: Regression Techniques in Machine Learning

Name: Kshitiz Reddy.M

Reg No.A26MTAI0008

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score

%matplotlib inline
plt.rcParams["figure.figsize"] = (7, 5)
np.random.seed(42)
print("Libraries loaded successfully.")
```

Libraries loaded successfully.

Question 1: Simple Linear Regression — Salary vs. Years of Experience

Objective: Predict `Salary` from `YearsExperience` using a simple linear regression model

```
# 1. Load the dataset
df1 = pd.read_csv("/content/Data/salary_data.csv")
print(df1.shape)
df1.head()
```

(30, 2)

	YearsExperience	Salary
0	1.1	39343
1	1.3	46205
2	1.5	37731
3	2.0	43525
4	2.2	39891

```
# 2. Train/test split (80/20)
X1 = df1[["YearsExperience"]].values
y1 = df1["Salary"].values

X1_train, X1_test, y1_train, y1_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42
)
print(f"Train size: {len(X1_train)}, Test size: {len(X1_test)}")
```

Train size: 24, Test size: 6

```
# 3-4. Fit the model and display intercept/slope
lr1 = LinearRegression()
lr1.fit(X1_train, y1_train)

beta0 = lr1.intercept_
beta1 = lr1.coef_[0]
print(f"Intercept ( $\beta_0$ ): {beta0:.2f}")
print(f"Slope ( $\beta_1$ ): {beta1:.2f}")
print(f"\nModel: Salary = {beta0:.2f} + {beta1:.2f} * YearsExperience")
```

Intercept (β_0): 25321.58
Slope (β_1): 9423.82

Model: Salary = 25321.58 + 9423.82 * YearsExperience

```
# 5. Predict salary for 7 years of experience
salary_7yr = lr1.predict([[7]])[0]
print(f"Predicted salary for 7 years of experience: ${salary_7yr:,.2f}")
```

Predicted salary for 7 years of experience: \$91,288.29

```
# 6. Evaluate on the test set
y1_pred = lr1.predict(X1_test)
r2_1 = r2_score(y1_test, y1_pred)
mse_1 = mean_squared_error(y1_test, y1_pred)
print(f"R² score: {r2_1:.4f}")
print(f"MSE: {mse_1:,.2f} (RMSE: ${np.sqrt(mse_1):,.2f})")
```

R² score: 0.9024
MSE: 49,830,096.86 (RMSE: \$7,059.04)

```
# 7. Plot the regression line over the training scatter
plt.figure()
plt.scatter(X1_train, y1_train, color="steelblue", label="Training data")
plt.scatter(X1_test, y1_test, color="crimson", marker="x", label="Test data")
order = np.argsort(X1_train[:, 0])
plt.plot(X1_train[order], lr1.predict(X1_train)[order], color="darkorange", linewidth=2, label="Regression line")
plt.xlabel("Years of Experience")
plt.ylabel("Salary ($)")
plt.title("Q1: Salary vs. Years of Experience")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



Question 2: Multiple Linear Regression — Predicting House Prices

```
# 8. Load the dataset
df2 = pd.read_csv("/content/Data/house_prices.csv")
print(df2.shape)
df2.head()
```

(50, 3)

	Size	Bedrooms	Price
0	1500	3	320000
1	2000	4	450000
2	1700	3	350000
3	1660	2	293168
4	2094	4	364752

```
# 9-10. Fit the model and display coefficients
X2 = df2[["Size", "Bedrooms"]].values
y2 = df2["Price"].values
```

```

X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42
)

lr2 = LinearRegression()
lr2.fit(X2_train, y2_train)

print(f"Intercept ( $\beta_0$ ): {lr2.intercept_:.2f}")
print(f"Coefficients [ $\beta_1$  (Size),  $\beta_2$  (Bedrooms)]: {lr2.coef_}")
print(f"\nModel: Price = {lr2.intercept_:.2f} + {lr2.coef_[0]:.2f}*Size + {lr2.coef_[1]:.2f}*Bedrooms")

```

Intercept (β_0): 51963.50
 Coefficients [β_1 (Size), β_2 (Bedrooms)]: [118.83218794 16545.38950033]

Model: Price = 51963.50 + 118.83*Size + 16545.39*Bedrooms

```

# 11. Predict price for Size=1800 sq.ft, Bedrooms=3
sample = np.array([[1800, 3]])
predicted_price = lr2.predict(sample)[0]
print(f"Predicted price for 1800 sq.ft, 3 bedrooms: ${predicted_price:,.2f}")

```

Predicted price for 1800 sq.ft, 3 bedrooms: \$315,497.61

```

# 12. Evaluate on held-out test set
y2_pred = lr2.predict(X2_test)
r2_2 = r2_score(y2_test, y2_pred)
print(f"R2 score on test set: {r2_2:.4f}")

```

R² score on test set: 0.9634

```

# 13. Plot the regression plane in 3D
fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(111, projection="3d")

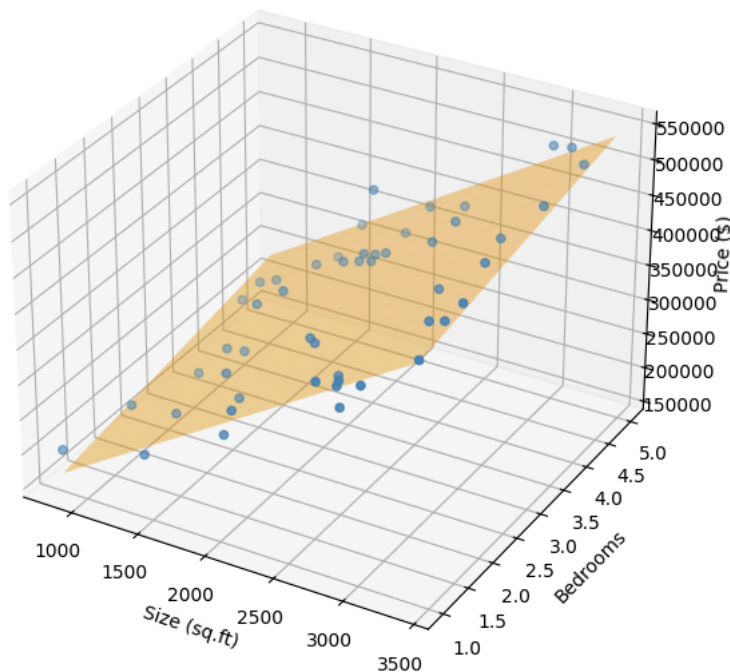
ax.scatter(df2["Size"], df2["Bedrooms"], df2["Price"], color="steelblue", label="Actual data")

size_range = np.linspace(df2["Size"].min(), df2["Size"].max(), 15)
bed_range = np.linspace(df2["Bedrooms"].min(), df2["Bedrooms"].max(), 15)
size_grid, bed_grid = np.meshgrid(size_range, bed_range)
price_grid = lr2.intercept_ + lr2.coef_[0] * size_grid + lr2.coef_[1] * bed_grid

ax.plot_surface(size_grid, bed_grid, price_grid, alpha=0.4, color="orange")
ax.set_xlabel("Size (sq.ft)")
ax.set_ylabel("Bedrooms")
ax.set_zlabel("Price ($)")
ax.set_title("Q2: Regression Plane — House Price")
plt.show()

```

Q2: Regression Plane — House Price



Question 3: Polynomial Regression — Modeling Plant Growth Curves

```
# 14. Load the dataset
df3 = pd.read_csv("/content/Data/plant_growth.csv")
print(df3.shape)
df3.head()
```

```
(30, 2)

   Days  Height
0     1     2.1
1     2     2.9
2     3     4.5
3     4     7.0
4     5     3.2
```

```
# 15. Fit a degree-3 polynomial regression (via pipeline)
X3 = df3[["Days"]].values
y3 = df3["Height"].values

poly3_model = make_pipeline(PolynomialFeatures(degree=3), LinearRegression())
poly3_model.fit(X3, y3)

lin_step = poly3_model.named_steps["linearregression"]
print("Polynomial (degree 3) coefficients [ $\beta_1$ ,  $\beta_2$ ,  $\beta_3$ ]:", lin_step.coef_[1:])
print("Intercept ( $\beta_0$ ):", lin_step.intercept_)
```

```
Polynomial (degree 3) coefficients [ $\beta_1$ ,  $\beta_2$ ,  $\beta_3$ ]: [-0.46925872  0.12440266 -0.00278617]
Intercept ( $\beta_0$ ): 3.6838934500991467
```

```
# 16. Fit a simple linear regression model for comparison
lin3_model = LinearRegression()
lin3_model.fit(X3, y3)

print(f"Linear model: Height = {lin3_model.intercept_:.3f} + {lin3_model.coef_[0]:.3f} * Days")
```

```
Linear model: Height = -0.024 + 1.004 * Days
```

```
# 17. Compare error metrics
y3_pred_lin = lin3_model.predict(X3)
y3_pred_poly = poly3_model.predict(X3)

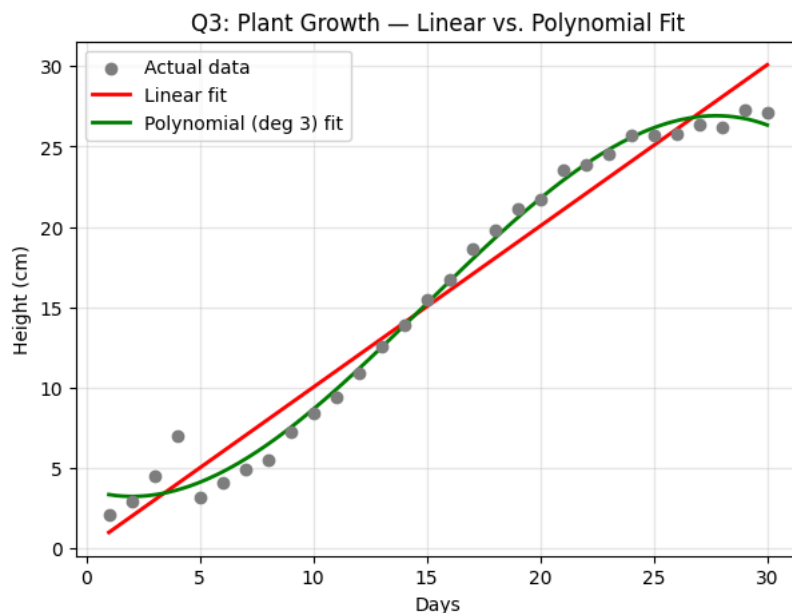
mse_lin = mean_squared_error(y3, y3_pred_lin)
mse_poly = mean_squared_error(y3, y3_pred_poly)
r2_lin = r2_score(y3, y3_pred_lin)
r2_poly = r2_score(y3, y3_pred_poly)

print(f"Linear      -> MSE: {mse_lin:.4f}, R²: {r2_lin:.4f}")
print(f"Polynomial3 -> MSE: {mse_poly:.4f}, R²: {r2_poly:.4f}")
```

```
Linear      -> MSE: 2.8238, R²: 0.9640
Polynomial3 -> MSE: 0.7152, R²: 0.9909
```

```
# 18. Plot both curves over the actual data
X3_smooth = np.linspace(X3.min(), X3.max(), 200).reshape(-1, 1)

plt.figure()
plt.scatter(X3, y3, color="gray", label="Actual data", zorder=5)
plt.plot(X3_smooth, lin3_model.predict(X3_smooth), color="red", linewidth=2, label="Linear fit")
plt.plot(X3_smooth, poly3_model.predict(X3_smooth), color="green", linewidth=2, label="Polynomial (deg 3) fit")
plt.xlabel("Days")
plt.ylabel("Height (cm)")
plt.title("Q3: Plant Growth – Linear vs. Polynomial Fit")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



Question 4: Polynomial Regression — Choosing the Right Degree (Bias–Variance Tradeoff)

Objective: Fit polynomial models of degree 1, 2, 4, and 8 to the position–salary dataset, and study underfitting vs. overfitting.

```
# 20. Load the dataset
df4 = pd.read_csv("/content/Data/position_salaries.csv")
df4
```

	Position	Level	Salary
0	Business Analyst	1	45000
1	Junior Consultant	2	50000
2	Senior Consultant	3	60000
3	Manager	4	80000
4	Country Manager	5	110000
5	Region Manager	6	150000
6	Partner	7	200000
7	Senior Partner	8	300000
8	C-level	9	500000
9	CEO	10	1000000

```
# 21-22. Fit polynomial models of increasing degree and record R²/MSE
```

```
X4 = df4[["Level"]].values
y4 = df4[["Salary"]].values
```

```
degrees = [1, 2, 4, 8]
poly_transformers = {}
poly_models = {}
metrics = {}
```

```
for d in degrees:
```

```
    poly = PolynomialFeatures(degree=d)
    X4_poly = poly.fit_transform(X4)
    model = LinearRegression()
    model.fit(X4_poly, y4)
```

```
    y4_pred = model.predict(X4_poly)
    r2 = r2_score(y4, y4_pred)
    mse = mean_squared_error(y4, y4_pred)
```

```
    poly_transformers[d] = poly
    poly_models[d] = model
    metrics[d] = (r2, mse)
    print(f"Degree {d:>2}: R² = {r2:.4f}, MSE = {mse:,.2f}")
```

```

Degree 1: R² = 0.6690, MSE = 26,695,878,787.88
Degree 2: R² = 0.9162, MSE = 6,758,833,333.33
Degree 4: R² = 0.9974, MSE = 210,343,822.84

```

```

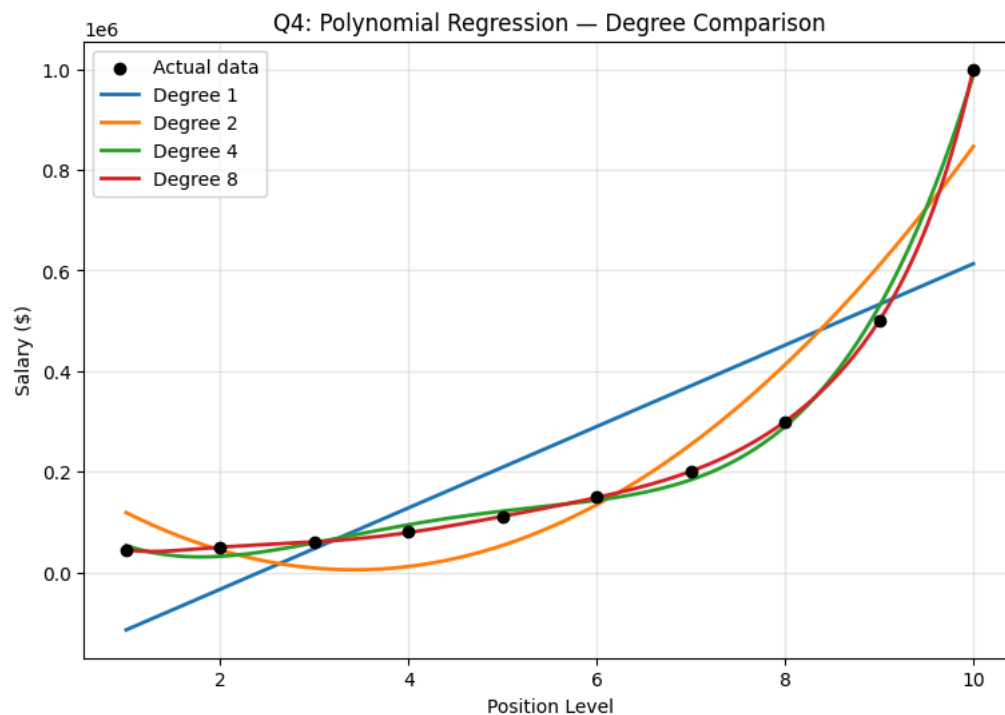
# 23. Plot all fitted curves together for visual comparison
X4_smooth = np.linspace(X4.min(), X4.max(), 200).reshape(-1, 1)

plt.figure(figsize=(9, 6))
plt.scatter(X4, y4, color="black", label="Actual data", zorder=5)

for d in degrees:
    poly = poly_transformers[d]
    model = poly_models[d]
    y_smooth = model.predict(poly.transform(X4_smooth))
    plt.plot(X4_smooth, y_smooth, linewidth=2, label=f"Degree {d}")

plt.xlabel("Position Level")
plt.ylabel("Salary ($)")
plt.title("Q4: Polynomial Regression — Degree Comparison")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```



```

# 24. Predict salary for Level = 6.5 using the best-performing model (degree 4)
best_degree = 4
poly_best = poly_transformers[best_degree]
model_best = poly_models[best_degree]

level_65 = poly_best.transform([[6.5]])
salary_65 = model_best.predict(level_65)[0]
print(f"Predicted salary at Level 6.5 (degree {best_degree} model): ${salary_65:,.2f}")

```

Predicted salary at Level 6.5 (degree 4 model): \$158,862.45